

Design and Analysis of Algorithms CoSc3012_

June 15, 2025

Compiled by: Natan Asrat

https://www.linkedin.com/in/natan_asrat

Design and Analysis of Algorithms

- Properties of an algorithm:
 - Finiteness: must complete after a finite number of steps.
 - Definiteness: each step must be clearly defined, having one and only one interpretation.
 - Sequence:
 - each step has unique preceding and succeeding steps.
 - first (start) and last (halt) step must be clearly noted.
 - Feasibility: must be possible to perform each instruction.
 - Correctness: must compute correct answer for all possible legal inputs.

Design and Analysis of Algorithms

- Properties of an algorithm:
 - Language Independence: must not depend on any one programming language.
 - Completeness: must solve the problem completely.
 - Effectiveness: must be possible to perform each step exactly and in a finite amount of time.
 - Efficiency: must solve with the least amount of computational resources.
 - Generality: should be valid on all possible inputs
 - Input/Output: must be a specified number of input values, and one or more result values.

Design and Analysis of Algorithms

- Analysis: provides a general idea of how long an algorithm will take for a given set
- Components of space complexity:
 - Instruction space: space needed to store the compiled version of the program instructions.
 - Factors:
 - Compiler used
 - Compiler options at time of compilation.
 - Target computer

Design and Analysis of Algorithms

- Components of space complexity:
 - Data space: space needed to store all constant and variable values.
 - Components:
 - Constants and simple variables.
 - Dynamically allocated objects such as arrays and class instances.

Design and Analysis of Algorithms

- Components of space complexity:
 - Environmental stack space: space used to save information needed to resume execution of partially completed methods (functions).
 - method func1 invokes or calls method func2, save a pointer where func1 will be executed when func2 terminates.
 - Each time a function or method is called or invoked, the following data are stored on the environment stack:
 - Return address
 - The values of all local variables and formal parameters in the method or function being invoked or called.

Design and Analysis of Algorithms

- Algorithm Analysis: produce a function $T(n)$ that describes the complexity of the algorithm
 - Order of Magnitude Analysis: Analysis of the function $T(n)$ to determine the general complexity category to which it belongs

Design and Analysis of Algorithms

- Algorithm Analysis:
 - No generally accepted set of rules
 - assume an arbitrary time unit
 - time 1 operations
 - Assignment Operation
 - Single Input/output Operation
 - Single Boolean Operations
 - Single Arithmetic Operations
 - Function Return
 - if/switch:
 - time for the condition evaluation plus
 - maximum of the running times for the individual clauses in the if/switch cases
 - loops:
 - running time of statements in the loop * number of iterations

Design and Analysis of Algorithms

- Measures of Times
 - Average Case: on an "average" set of inputs.
 - Worst Case: on the worst possible set of inputs.
 - Best Case: on the smallest possible set of inputs.

Design and Analysis of Algorithms

- Design and analysis of Divide and Conquer Algorithms
 - Base case scenario OR you divide the problem, run algorithms C & D on the subproblems and return the combined result of C & D
 - The complexity: (a - number of subproblems, n/b - input size of subproblems D & C - algorithms that take care of subproblems)

Design and Analysis of Algorithms

- Greedy Model: most straightforward designed technique, 'short sighted', and 'non-recoverable'
 - ideal only for problems which have 'optimal substructure'.
 - best suited for simple problems
 - mostly (but not always) fail to find the globally optimal solution
- Variations:
 - Pure greedy algorithms
 - Orthogonal greedy algorithms
 - Relaxed greedy algorithms

Design and Analysis of Algorithms

- Greedy Model:

- Knapsack Problem

- given n objects and knapsack or bag with capacity M
 - objects have weights and profits
 - we have to fill the bag with the help of N objects and the resulting profit has to be maximum
 - Steps:
 - sort objects by profit/weight ratio in descending order
 - select highest p/w ratio objects if their weights are less than the bag's available weight and decrement available weight by the selected object's weight and repeat

Design and Analysis of Algorithms

- Greedy Model:
 - Job Sequencing with Deadlines
 - given number of jobs (with profit and deadlines)
 - find sequence of jobs completed within deadlines yielding maximum profit
 - waiting time and the processing time should be less than or equal to the deadline of the job

Design and Analysis of Algorithms

- Greedy Model:
 - Minimum Cost Spanning Tree
 - A sub-graph $t=(V,E)$ of the G is a Spanning tree of G iff t is a tree
 - generate a graph $G = (V,E)$ where
 - We have to visit all the nodes
 - does not form a cycle
 - any connected graph with N vertices must have at least $N-1$ edges

Design and Analysis of Algorithms

- Greedy Model – Minimum Cost Spanning Tree
 - Algorithms
 - Prim's Algorithm: compare's edge's weights before including them
 - start from an arbitrary vertex (root)
 - select the smallest edge that connects to the already chosen vertices
 - Kruskal's Algorithm:
 - Sort all edges in increasing order of weight.
 - Pick the smallest edge and add it to the MST, as long as it doesn't form a cycle
 - result is a forest of trees that grows until all the trees merge in a single tree

Design and Analysis of Algorithms

- Greedy Model:
 - Optimal Merge Patterns: merging two sorted files into one sorted file
 - Pair wise merging:
 - have pointers on both linked lists first elements, pointers: p1 on LL1, p2 on LL2
 - have another pointer curr on the smallest of the two first items of the linked lists
 - advance p1 if it is smaller or advance p2 otherwise
 - compare p1 and p2's values and connect the next of curr to the smallest, advance the smallest from p1 and p2, advance curr to the next (smallest chosen), repeat

Design and Analysis of Algorithms

- Greedy Model:
 - Optimal Merge Patterns:
 - For multiple files:
 - Merge two smallest available files together
 - On the next merge, use the merged file with other available smallest files or just use the other smallest available files

Design and Analysis of Algorithms

- Greedy Model – Optimal Merge Patterns:
 - Huffman encoding
 - https://www.youtube.com/watch?v=co4_ahEDCho
 - reduce number of bits required using new encoding instead of ASCII for only the characters you have used... i.e. instead of 7 bits, you can have 3 bits if you use less than 8 characters, and 2 bits if you have less than 4 characters in the set of characters for your message.
 - Binary 'a': **01100001** , Binary 'b': **01100010**, Binary 'c': **01100011**
 - Your encoding for 'a': 00, your 'b': 01, your 'c': 10

Design and Analysis of Algorithms

- Greedy Model – Optimal Merge Patterns:
 - Huffman encoding
 - Huffman's approach:
 - rearrange the characters by frequency of occurrence
 - use optimal merge to construct the binary tree like the above image
 - the left edges will be labeled as 0 and right edges as 1
 - character (leaf) will have the encoding same as the labels in path from the root – shorter path will have less number of bits (inconsistent – sometimes 2 bits sometimes 3 bits per character ...)
 - use the tree to decode by traversing, write character when reached leaf, and start from root for the rest and repeat

Design and Analysis of Algorithms

- Greedy Model:
 - Single Source Shortest Path Algorithm (Dijkstra's)
 - Single-source shortest path:
 - length of a path is now defined to be the sum of the weights of the edges on that path
 - determine the shortest path from v_0 to all the remaining vertices of G

Design and Analysis of Algorithms

- Greedy Model – Single Source Shortest Path Algorithm (Dijkstra's):
 - Dijkstra's Algorithm:
 - <https://www.youtube.com/watch?v=XB4MIexjvY0>
 - set distance for directly connected nodes to the source vertex (length of path)
 - set other node's distance to infinity
 - choose the smallest distance as current node and calculate the distance for the connected nodes by adding the distance of current node to the length of the edges connecting them.
 - if the calculated distance is less than the current distance, reassign (relaxation) – initially infinity so any distance will do on first time, on other visits it may or may not be reassigned, repeat above step only choosing unselected (smallest distance) nodes.

Design and Analysis of Algorithms

- Dynamic programming
 - like the divide-and-conquer method, solves problems by combining the solutions to subproblems
 - unlike divide-and-conquer, which solves the subproblems top-down, a dynamic programming is a bottom-up technique
 - Start with the smallest sub problems and combine solutions
 - stage-wise search suitable for optimization problems solved with a sequence of decisions
 - early partial decision solutions are reused by keeping a table of known results of subproblems
 - Principle of Optimality: the optimal solution to a problem is a combination of optimal solutions to some of its sub problems

Design and Analysis of Algorithms

- Dynamic programming
 - Multistage Graph: $k \geq 2$ disjoint sets of graphs (no edge in the same set, but edges connect nodes from one set to another); k = number of sets
 - problem: find minimum cost path from source node to destination node
 - cost matrix: stores the distance from any vertex to the destination

Design and Analysis of Algorithms

- Dynamic programming – Multistage Graph:
 - Solution:
 - $\text{cost}(i, j)$ – cost of node at stage i and vertex j
 - $\text{cst}(i, j)$ or $c(i, j)$ – cost (length of path if direct) between vertex i and vertex j

Design and Analysis of Algorithms

- Dynamic programming – Multistage Graph:
 - Solution – forward method:
 - <https://www.youtube.com/watch?v=9iE9Mj4m8jk>
 - start from last node and calculate the distance from each node connected to the last node (the previous stage), save the cost for each node and which path it took (path to destination node is the only path here)
 - to calculate for the nodes in the previous stage from there, calculate the minimum cost from each node to destination = length of edge to node in next stage + cost of node on the next stage (already calculated cost so reuse that value)... choose from the edges of that node in your current stage to find minimum cost for that node, and calculate for the other nodes in that stage
 - go back one more stage and repeat until reached the source node

Design and Analysis of Algorithms

- Dynamic programming – Multistage Graph:
 - Solution – backward method:
 - <https://www.youtube.com/watch?v=JHRuqckZxWk>
 - start from source node and set its cost to 0, calculate the next stage's nodes cost which is the length of path from source the nodes
 - on the next stage after that, calculate cost of nodes by getting the minimum cost from their edges to the previous nodes (i.e. if node N4 has two edges to nodes N2 and N3, the minimum is $\min(\text{cost}(2,4) + \text{cost}(1,2), \text{cost}(3,4) + \text{cost}(1,3))$; or length of path to previous node + cost of that previous node (the minimum of all edges).
 - go forward one more stage and calculate the next nodes until you reach the destination node

Design and Analysis of Algorithms

- Dynamic programming:
 - All Pair Shortest Path
 - <https://www.youtube.com/watch?v=oNI0rf2P9gE>
 - calculate distance from each node to other node
 - Dijkstra only finds shortest path between two nodes
 - first build a table/matrix A_0 where rows and columns are nodes and cells are the direct length from node in the row to node in the column, if no direct edge set the cell to infinity

Design and Analysis of Algorithms

- Dynamic programming - All Pair Shortest Path
 - now build another table/matrix A_1 where the node 1 (1st row; also 1st column) is used as an intermediary node, so 1st row and 1st column will not be changed,
 - calculate the remaining cells by comparing the cell value in A_0 to what the cell value would be if node 1 is intermediary and take the minimum
 - i.e. for row 2, column 3 $\Rightarrow (2, 3)$, assume the cell $A_0(2, 3)$ has value 4 and since we are using node intermediary we will add $A_0(2, 1) + A_0(1, 3)$... assume $A_0(2, 1) = 1$, $A_0(1, 3) = 2$; $A_0(2, 1) + A_0(1, 3) = 1 + 2 = 3 < A_0(2, 3)$ so we choose the intermediary since its smaller (the path from 2 to 3 through 1 is shorter than the direct path from 2 to 3)
 - when building the second table having second node as intermediary use A_1 as reference instead of A_0

Design and Analysis of Algorithms

- Dynamic programming - All Pair Shortest Path
 - General formula:
 - $A^k[i, j] = \min(A^{k-1}[i, k], A^{k-1}[k, j])$

Design and Analysis of Algorithms

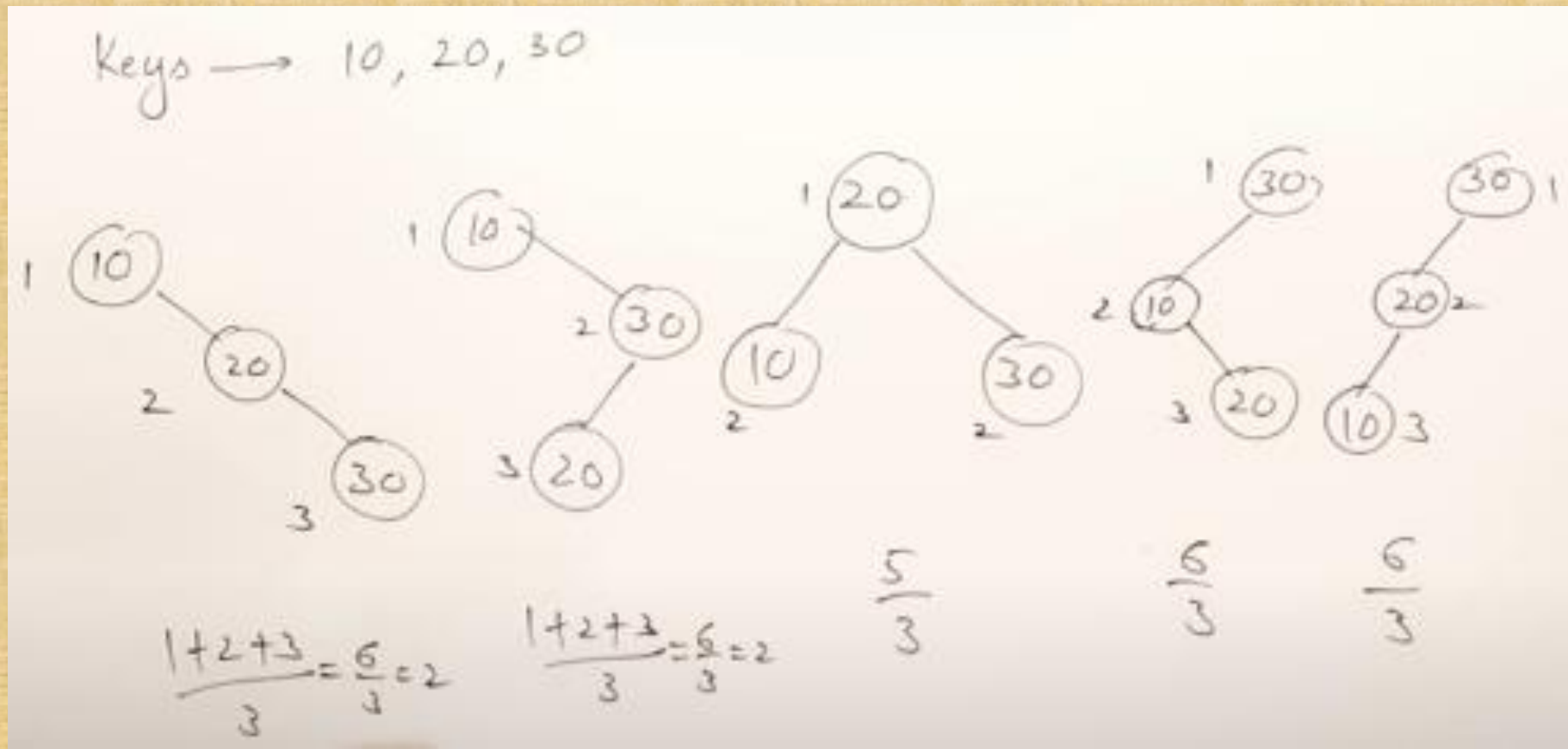
- Dynamic programming
 - Optimal Binary Search Trees
 - <https://www.youtube.com/watch?v=vLS-zRCHo-Y>
 - in an optimal tree, all the subtrees must also be optimal with respect to the keys that they contain
 - for a sorted array of keys, there could be different variations of binary search trees built
 - the optimal binary search tree is one with least cost or less number of comparisons to make when searching if an element exists in a tree (if searching for a greater value you choose right branch, else you look in left branch)

Design and Analysis of Algorithms

- Dynamic programming - Optimal Binary Search Trees
 - the search terminates when reached the leaf without finding the element so the deeper the binary search tree the more comparisons are made and the cost will be more
 - if the tree is optimized into having less depth and more branches, the comparison/cost will be less since the other branch will not be checked if one branch is chosen according to the comparison to the parent node

Design and Analysis of Algorithms

- Dynamic programming - Optimal Binary Search Trees



Design and Analysis of Algorithms

- Dynamic programming
 - 0/1 Knapsack problem
 - 0/1 : either carry whole object or dont, not divisible
 - maximize profit, weight of carried objects $\leq$ weight capacity of bag
 - Time complexity: $O(n * W)$.
 - n is the number of items and
 - W is the weight of the knapsack

Design and Analysis of Algorithms

- Dynamic programming - 0/1 Knapsack problem
 - Steps
 - <https://www.youtube.com/watch?v=nLmhmB6NzcM>
 - Make a table with $n+1$ rows and $W+1$ columns (n - number of items, W - capacity of bag)
 - first row and first column are 0 (first row - no item, first column - no weight used)
 - in the second row cells (first item) write the profit (cell value) as 0 if weight of item is more than column (weight of knapsack), else write profit of item in the cells where the item's weight is equal to or less than the column

Design and Analysis of Algorithms

- Dynamic programming - 0/1 Knapsack problem
 - Steps
 - in the next rows, fill the above row's values where the weight of item is more than the column, on first equal encounter fill the item's weight, on next column for the row, check if it is possible to combine any of the above rows' items and if so write the combined profit
 - After filling the table, find the maximum value of profit, check if the above rows have the maximum value in any of the columns, pick the top most row with that maximum value and include that item in knapsack, i.e. 8 (maximum), item 4
 - Deduct the profit of picked item from the maximum profit and look for the top most row with that value (i.e. item 4's profit = 6, deduct: $8 - 6 = 2$, look for 2 in the rows; item 2 & 3 have value 2 in their rows, pick 2; deduct; $2 - 2 = 0$... finished with item 4 and 2)

Design and Analysis of Algorithms

- Dynamic programming - 0/1 Knapsack problem
 - Steps - General formula:
 - $A[i, w] = \max \{ A[i-1, w], A[i-1, w-W[i]] + P[i] \}$
 - i - item row; w - knapsack weight column;
 - $W[i]$ - weight of item, $P[i]$ - profit of item
 - second part in max: $A[i-1, w-W[i]] + P[i]$ could be invalid if $w-W[i]$ is negative or $W[i] > w$ (weight of item is more than knapsack so it wont fit), in that case you are using the above row's value (left part of max) $A[i-1, w]$

Design and Analysis of Algorithms

- Dynamic programming - 0/1 Knapsack problem
 - General formula: $fn(M) = \text{Max} \{ fn-1(x), fn-1(M - Wn) + Pn \}$

$m=8$
 $n=4$

$P = \{1, 2, 5, 6\}$
 $W = \{2, 3, 4, 5\}$

		W									
		V	0	1	2	3	4	5	6	7	8
P_i	W_i	0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

$V[i, w] = \max \{ V[i-1, w], V[i-1, w - W_i] + P_i \}$
 $V[4, 8] = \max \{ V[3, 8], V[3, 8-5] + 6 \}$
 $7, 2+6$

Design and Analysis of Algorithms

- Dynamic programming
 - Reliability Design
 - <https://www.youtube.com/watch?v=uJ0mqBwENB8>
 - problem: in a system, some devices may fail and you will need extra just in case, reliability of each device is likelihood it will continue working, reliability of system is product of all it's component's reliability.. task is to increase reliability by keeping multiple copies of a device with cost in mind and finding an optimum solution with highest system reliability

Design and Analysis of Algorithms

- Dynamic programming - Reliability Design
 - Solution:
 - set a budget, i.e. 105, and calculate how much it takes to buy 1 of each device... calculate the remaining cash and with it you can buy more devices
 - upper bound for a device is how many you can buy with the remaining cash after you bought 1 of each device (includes 1 that you bought first then add the floor of remaining cash/cost of device]

D_i	C_i	r_i	u_i
D_1	30	0.9	2
D_2	15	0.8	3
D_3	20	0.5	3

Design and Analysis of Algorithms

- Dynamic programming - Reliability Design
 - Solution:
 - create set $S^0 = \{(1, 0)\}$ which means when no device is bought and cost is 0, reliability is 1
 - create next set for first device
 - when buying only one copy of device 1
$$S_1^1 = \{(0.9, 30)\}$$
 - when buying two copies of device 1
$$S_2^1 = \{(0.99, 60)\}$$
 - $S^1 = \{(0.9, 30), (0.99, 60)\}$ - combined the two sets for device 1
 - superscript - device # ; subscript - # of copies
 - reliability for multiple copies:
 - $1 - (1 - r)^k$ where k is # of copies, r is reliability of a device

D_i	C_i	r_i	u_i
D_1	30	0.9	2
D_2	15	0.8	3
D_3	20	0.5	3

$$\begin{aligned} 1 - (1 - r_1)^2 &= 1 - (1 - 0.9)^2 \\ &= 1 - (0.1)^2 \\ &= 1 - 0.01 \\ &= 0.99 \end{aligned}$$

Design and Analysis of Algorithms

- Dynamic programming - Reliability Design
 - Solution:
 - on the next set, multiply the reliability of a device (# of copies changes this) with the previous reliabilities
 - when buying only one copy of device 2
 - device 1 of 1 copy $S_1^2 = \{(0.8 *$

Design and Analysis of Algorithms

• Dynamic programming - Reliability Design

• Solution:

- on the next set, multiply the reliability of a device (# of copies changes this) with the previous reliabilities

• when buying two copies of device 2

- device 1 of 1 copy

$$\begin{aligned} S_2^2 &= \{(0.96 * 0.9, 30 + 30)\} \\ &= \{(0.86, 60)\} \end{aligned}$$

- device 1 of 2 copies

$$\begin{aligned} S_2^2 &= \{(0.96 * 0.99, 60 + 30)\} \\ &= \{(0.95, 90)\} \end{aligned}$$

- discard second set since for cost 90 currently, we cant buy device 3 for 20 and still have a budget of 105: $S^2 = \{(0.86, 60)\}$

D_i	C_i	r_i	u_i
D_1	30	0.9	2
D_2	15	0.8	3
D_3	20	0.5	3

$C = 105$

$r_3 = 0.5$

$1 - (1 - r_3)^3$

$1 - (1 - 0.5)^3$

$1 - (0.5)^3$

$S^0 = \{(1, 0)\}$

$D_1 = 1$

$S_1^1 = \{(0.9, 30)\}$

$S_2^1 = \{(0.99, 60)\}$

$S^1 = \{(0.9, 30), (0.99, 60)\}$

$D_2 = 2$

$S_1^2 = \{(0.72, 45), (0.792, 75)\}$

$S_2^2 = \{(0.864, 60), (0.9, 90)\}$

$S_3^2 = \{(0.8928, 75), (0.9, 105)\}$

$S^2 = \{(0.72, 45), (0.792, 75), (0.864, 60), (0.8928, 75)\}$

$D_3 = 2$

$S_1^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$

$S_2^3 = \{(0.54, 85), (0.648, 100), (0.6, 115)\}$

$S_3^3 = \{(0.63, 105), (0.72, 120), (0.7135, 135)\}$

$S^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95), (0.54, 85), (0.6, 105), (0.648, 100)\}$

Design and Analysis of Algorithms

• Dynamic programming - Reliability Design

• Solution:

- remove pairs in the set where the next item has more reliability with less cost

• i.e.

$$S = \{(0.72, 45), (0.79, 75), (0.86, 60), (0.89, 75)\}$$

- remove second element since the next element has more reliability with less cost

$$S = \{(0.72, 45), (0.86, 60), (0.89, 75)\}$$

D_i	C_i	r_i	u_i
D_1	30	0.9	2
D_2	15	0.8	3
D_3	20	0.5	3

$C = 105$

$r_3 = 0.5$

$1 - (1 - r_3)^3$

$1 - (1 - 0.5)^3$

$1 - (0.5)^3$

Handwritten calculations for dynamic programming:

$S^0 = \{(1, 0)\}$

$D_1 = 1$

$S'_1 = \{(0.9, 30)\}$

$S'_2 = \{(0.99, 60)\}$

$S' = \{(0.9, 30), (0.99, 60)\}$

$D_2 = 2$

$S^2_1 = \{(0.72, 45), (0.792, 75)\}$

$S^2_2 = \{(0.864, 60), (\text{crossed out}, 90)\}$

$S^2_3 = \{(0.8928, 75), (\text{crossed out}, 105)\}$

$S^2 = \{(0.72, 45), (0.792, 75), (0.864, 60), (0.8928, 75)\}$

$D_3 = 2$

$S^3_1 = \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$

$S^3_2 = \{(0.54, 85), (0.648, 100), (\text{crossed out}, 115)\}$

$S^3_3 = \{(0.63, 105), (\text{crossed out}, 120), (\text{crossed out}, 135)\}$

$S^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95), (0.54, 85), (0.648, 100)\}$

Design and Analysis of Algorithms

• Dynamic programming - Reliability Design

• Solution:

- continue until the set for the last item is complete and choose the highest reliability
- trace back the origins for that set element to find optimum count of devices

D_i	C_i	r_i	u_i
D_1	30	0.9	2
D_2	15	0.8	3
D_3	20	0.5	3

$C = 105$

$r_3 = 0.5$

$S^0 = \{(1, 0)\}$
 $D_1 = 1$
 $S_1^1 = \{(0.9, 30)\}$
 $S_2^1 = \{(0.99, 60)\}$
 $S^1 = \{(0.9, 30), (0.99, 60)\}$
 $D_2 = 2$
 $S_1^2 = \{(0.72, 45), (0.792, 75)\}$
 $S_2^2 = \{(0.864, 60), (\text{---}, 90)\}$
 $S_3^2 = \{(0.8928, 75), (\text{---}, 105)\}$
 $S^2 = \{(0.72, 45), (0.792, 75), (0.864, 60), (0.8928, 75)\}$
 $D_3 = 2$
 $S_1^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$
 $S_2^3 = \{(0.54, 85), (0.648, 100), (\text{---}, 115)\}$
 $S_3^3 = \{(0.63, 105), (\text{---}, 120), (\text{---}, 135)\}$
 $S^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95), (0.54, 85), (0.63, 105), (0.648, 100)\}$

$1 - (1 - r_3)^3$
 $1 - (1 - 0.5)^3$
 $1 - (0.5)^3$

(0.648, 100)

Design and Analysis of Algorithms

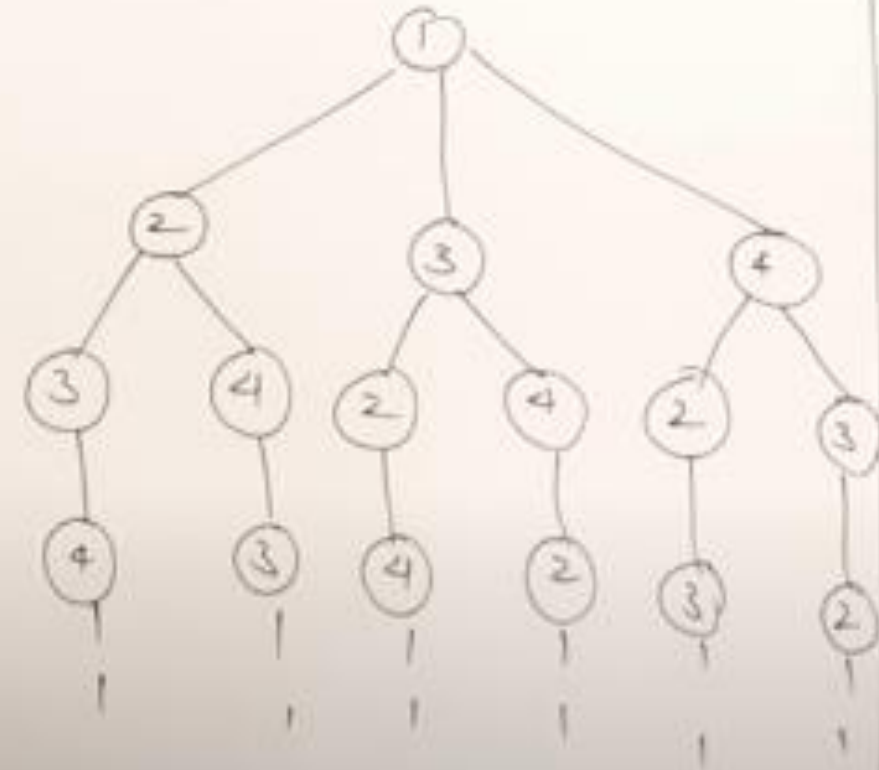
- Dynamic programming:
 - Travelling Salesman Problem:
 - <https://www.youtube.com/watch?v=XaXsJJh-Q5Y>
 - directed graph with edge cost c_{ij} is defined, find a tour of minimum cost
 - tour of G is a directed cycle that include every vertex in V = shortest path that starts and ends at the same vertex

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 10 & 15 & 20 \\ 5 & 0 & 9 & 10 \\ 6 & 13 & 0 & 12 \\ 8 & 8 & 9 & 0 \end{bmatrix} \end{matrix}$$



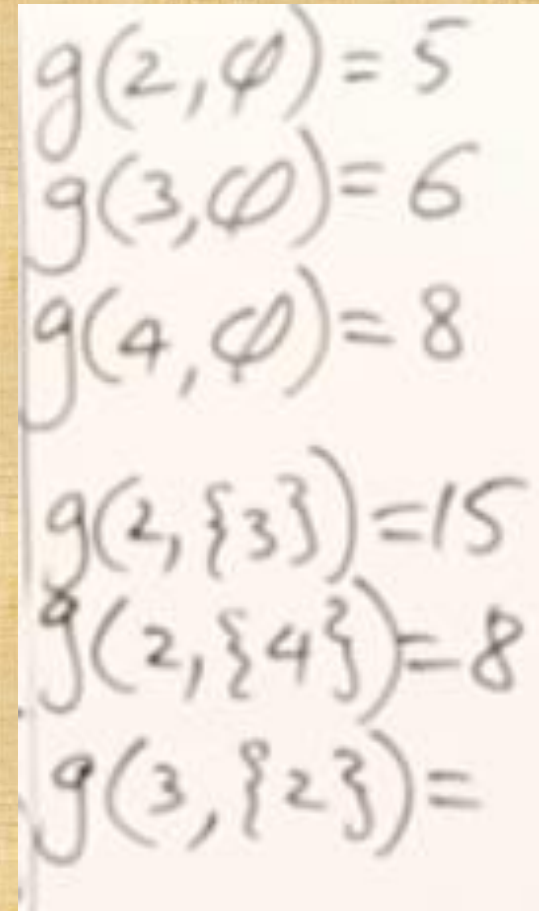
Design and Analysis of Algorithms

- Travelling Salesman Problem:
 - Brute force
 - draw a tree where the root is the start node
 - connect children nodes of all remaining available nodes except starting node
 - continue with 3rd level where children of a 2nd level node are from available nodes except root and 2nd level node itself, continue downward in same fashion



Design and Analysis of Algorithms

- Travelling Salesman Problem:
 - Dynamic programming
 - General formula
 - $g(i, k) = \min_{k \in S} \{c_{i,k} + g(k, S - \{k\})\}$
 - express the costs according with $g(i, \{...\})$ and reuse that cost instead of recalculating/fetching from matrix
 - $g(2, \alpha)$ means from node 2 back to source (1)
 - $g(2, \{3\})$ means being on node 2 where remaining node is 3 (in bottom-up it means you have visited 3 and you are now at 2)



Handwritten notes on a yellow background showing dynamic programming values for the Traveling Salesman Problem:

$$\begin{aligned}g(2, \emptyset) &= 5 \\g(3, \emptyset) &= 6 \\g(4, \emptyset) &= 8 \\g(2, \{3\}) &= 15 \\g(2, \{4\}) &= 8 \\g(3, \{2\}) &= \end{aligned}$$

Design and Analysis of Algorithms

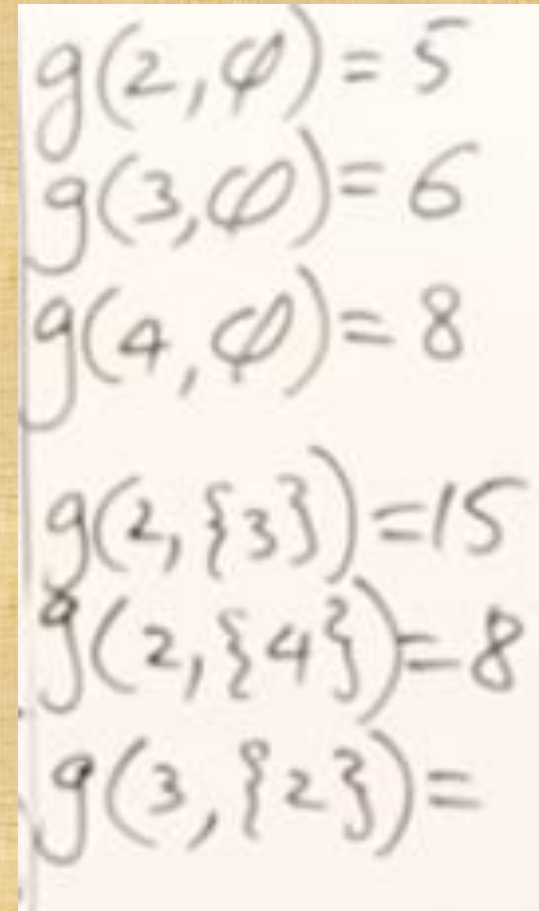
- Travelling Salesman Problem:
 - Dynamic programming
 - General formula
 - $g(i, k) = \min_{k \in S} \{c_{i,k} + g(k, S - \{k\})\}$
 - express the costs according with $g(i, \{...\})$ and reuse that cost instead of recalculating/fetching from matrix
 - $g(2, \{3, 4\})$ means 3 and 4 are remaining (in bottom-up you visited either 3 and then 4, or it means you visited 4 and then 3 but you are now in node 2)
 - $4 \rightarrow 3 \rightarrow 2$ scenario
 - $3 \rightarrow 4 \rightarrow 2$ scenario
 - choose minimum
 - $g(2, \{3, 4\}) = \min(c_{2,3} + g(3, \{4\}), c_{2,4} + g(4, \{3\}))$ - the values inside have already been calculated, multiply by either $c_{2,3}$ or $c_{2,4}$ and get the min
 - $g(2, \{3, 4\}) = \min(c_{2,3} + g(2, \{4\}), c_{2,4} + g(2, \{3\}))$

Handwritten notes showing recursive calculations for the Traveling Salesman Problem using dynamic programming:

$$\begin{aligned}g(2, \emptyset) &= 5 \\g(3, \emptyset) &= 6 \\g(4, \emptyset) &= 8 \\g(2, \{3\}) &= 15 \\g(2, \{4\}) &= 8 \\g(3, \{2\}) &= \end{aligned}$$

Design and Analysis of Algorithms

- Travelling Salesman Problem:
 - Dynamic programming
 - General formula
 - $g(i, k) = \min_{k \in S} \{c_{i,k} + g(k, S - \{k\})\}$
 - express the costs according with $g(i, \{...\})$ and reuse that cost instead of recalculating/fetching from matrix
 - finally calculate
$$g(1, \{2, 3, 4\}) = \min(\begin{aligned} &c_{1,2} + g(2, \{3, 4\}), \\ &c_{1,3} + g(3, \{2, 4\}), \\ &c_{1,4} + g(4, \{2, 3\}) \end{aligned})$$



Handwritten notes showing values for the g function in the TSP dynamic programming algorithm:

- $g(2, \emptyset) = 5$
- $g(3, \emptyset) = 6$
- $g(4, \emptyset) = 8$
- $g(2, \{3\}) = 15$
- $g(2, \{4\}) = 8$
- $g(3, \{2\}) =$

Design and Analysis of Algorithms

- Traversal Techniques
 - Complete graph: undirected graph with
 - n vertices
 - exactly $n(n-1)/2$ edges
 - Reachability problem: determine whether there exist a path in the graph which starts at vertex ' v ' and ends at vertex ' u '

Design and Analysis of Algorithms

- Traversal Techniques

- Search methods:

- <https://www.youtube.com/watch?v=pcKY4hjDrxk>

- Breadth First: uses queue of unexplored nodes and array of visited nodes

- start at vertex v and mark it as having been reached
 - vertex is 'explored' when the algorithm has visited all vertices adjacent from it
 - all unvisited vertices adjacent from v are visited next
 - there are new unexplored vertices (children or next level)
 - continues until no unexplored vertex is left
 - time complexity becomes $O(n^2)$ if adjacency list and adjacency matrix is used, else $O(n+e)$ - n is number of vertices, e is number of edges
 - space complexity - $O(n)$

Design and Analysis of Algorithms

- Traversal Techniques

- Search methods:

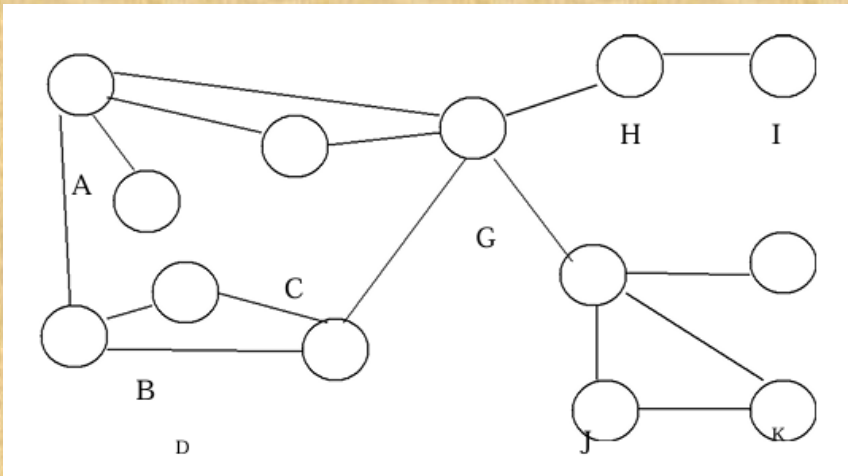
- Depth First: uses a stack (explicit stack or recursion) for unexplored nodes and an array of visited nodes
 - exploration of a vertex v is suspended as soon as a new vertex(child) is reached
 - exploration of the new vertex(child) u begins
 - search terminates when all reached vertices have been fully explored
 - same complexity as BFS

Design and Analysis of Algorithms

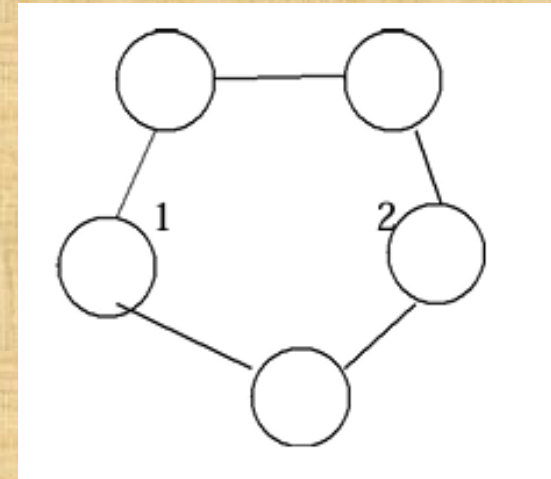
- Traversal Techniques

- Bi-connected:

- Articulation Point: a vertex that if deleted, would break the graph into two or more pieces (Connected components)
 - i.e. A, G, H, J



- Bi-connected graph: has no articulation points; if any vertex is deleted the graph remains connected
 - Bi-connected component: subgraph that is bi-connected



Design and Analysis of Algorithms

- Backtracking
 - <https://www.youtube.com/watch?v=DKCbsiDBN6c>
 - one of the most general algorithm design techniques
 - many searching and optimization problems satisfying some constraints can be solved with backtracking
 - the problem is to find a vector, which maximizes or minimizes a criterion function $P(x_1 \dots x_n)$
 - Constraints:
 - Explicit constraints: rules that restrict each X_i to take values only from a given set; i.e. $X_i > 0$
 - Implicit constraint: determines which of the vectors actually satisfy the criterion functions

Design and Analysis of Algorithms

- Backtracking – Examples:
 - N-Queens Problem: place n -queens in an N by N matrix in such a way that no two queens attack each (no two queens should be in the same row, column, diagonal)
 - https://www.youtube.com/watch?v=xFv_Hl4B83A
 - Sum of Subsets: find all the combination of numbers from a set of positive numbers that sum up to a number N
 - <https://www.youtube.com/watch?v=kyLxTdsT8ws>

Design and Analysis of Algorithms

- Backtracking – Examples:
 - Graph Coloring: color a graph G in such a way that no two adjacent nodes have the same color with a limited number of m (count) available colors
 - Problems:
 - m -coloring optimization: coloring a graph using the minimum number of coloring.
 - m -coloring decision(K -coloring): whether graph is colorable using at most k -colors
 - <https://www.youtube.com/watch?v=052VkKhIaQ4>
 - Hamiltonian cycle: a round trip path along ' n ' edges of graph G which visits every vertex once and returns to its starting position
 - <https://www.youtube.com/watch?v=dQr4wZCiJJ4>

Design and Analysis of Algorithms

- Backtracking – Examples:
 - Branch and Bound
 - https://www.youtube.com/watch?v=3RBNPc0_Q6g
 - for solving combinatorial optimization problems
 - Branching: Dividing the problem into smaller subproblems (like a decision tree).
 - Bounding: Estimating the best possible solution for each subproblem.
 - Pruning: Skipping branches that can't lead to a better solution than the current best.

Design and Analysis of Algorithms

- Backtracking – Examples:
 - Knapsack Problem using Backtracking: similar to the (0/1) knapsack
 - https://www.youtube.com/watch?v=yV1d-b_NeK8
 - The Solution space is the same as that for the sum of subset's problem.
 - Bounding functions are needed to help kill some live nodes without expanding them
 - Travelling Salesman Problem: minimization problem, shortest possible route that visits each city once and returns?
 - solved using Branch and bound by generating state space tree by calculating cost and upper bound